

Technical Blueprint: Dhanvantari

Matrimony Platform

Architecture Model: Edge-Deployed Serverless Single Page Application (SPA)

Infrastructure Provider: Cloudflare (Pages, Workers, D1, R2)

1. Executive Summary

Unlike traditional monolithic CMS platforms (like WordPress) which rely on heavy PHP processing, a centralized MySQL database, and bloated plugins, this platform utilizes a **Modern Serverless Edge Architecture**.

By decoupling the frontend interface from the backend database and deploying the code directly to Cloudflare's global edge network, the platform achieves near-instantaneous load times, infinite horizontal scalability, and zero-maintenance server infrastructure. The site functions and feels like a native mobile application while remaining accessible via standard web browsers.

2. Frontend Architecture (The Client Layer)

The frontend is engineered for maximum speed and zero bloat, completely avoiding heavy Javascript frameworks (like React or Angular) in favor of highly optimized Vanilla Web Technologies.

- **Tech Stack:** HTML5, CSS3 (CSS Variables, Flexbox, Grid), Vanilla ES6 JavaScript.
- **Native-App Routing (History API):** Instead of reloading the page every time a user clicks a profile, the platform uses asynchronous JavaScript (AJAX) to load data into a floating Modal. It intercepts the Android/iOS native "Back" buttons using the HTML5 `history.pushState` API, allowing users to seamlessly close profiles without losing their place in the feed.
- **Dynamic DOM Manipulation:** The UI renders dynamically based on the JSON payload received from the API. Filtering (by Name, Age, Gender) happens instantly in the browser's memory without requiring additional database queries, ensuring a zero-latency search experience.

- **Client-Side Compute (Image Pipeline):** To save bandwidth and storage, the platform utilizes the client's device (smartphone/PC) to process images. Using `compressor.js`, heavy 10MB+ camera photos are resized to HD dimensions and compressed to under 500KB *before* they even hit the network.
-

3. Backend Architecture (The Edge API Layer)

The backend does not exist on a traditional server. Instead, it utilizes **Cloudflare Pages Functions** (Serverless Workers) which spin up in milliseconds on servers physically located closest to the user.

- **RESTful API Design:** The backend exposes stateless endpoints (`/api/profiles` , `/api/upload` , `/api/auth`) utilizing standard HTTP methods:
 - `GET` : Fetch profiles (paginated).
 - `POST` : Submit new profiles (forces 'pending' status) / Upload images.
 - `PUT` : Update existing profiles / Approve pending profiles.
 - `DELETE` : Remove profiles and trigger R2 object deletion.
 - **Pagination Engine:** The API protects database resources by serving profiles in chunks (20 per page) using SQL `LIMIT` and `OFFSET` commands, requested via a "Load More" interface.
 - **Orphaned File Prevention:** When a `DELETE` request is authorized, the API first queries the database for the image URLs, sends a deletion command to the R2 storage bucket to wipe the files, and *then* deletes the database row. This keeps storage costs permanently optimized.
-

4. Data Persistence (Database & Storage)

The data layer is split into two highly specialized, decoupled Cloudflare services:

- **Relational Database (Cloudflare D1):**
 - Built on SQLite, distributed globally.
 - Maintains a strict schema containing demographic, astrological, professional, and family data.
 - Tracks entity state via a `status` column (`pending` vs. `approved`).
- **Object Storage (Cloudflare R2):**
 - An S3-compatible blob storage bucket used exclusively for user media.
 - Unlike traditional web hosts, R2 charges zero egress (bandwidth) fees, meaning millions of people can view the HD profile photos without incurring bandwidth penalties.

5. Security & Privacy Model

Because matrimonial platforms handle Personally Identifiable Information (PII), the platform implements a strict, multi-layered security model.

- **Data Masking (Over-fetching Prevention):** The `/api/profiles` endpoint intelligently checks the user's authorization.
 - *Public Requests:* Receives a stripped-down JSON payload (Name, Age, Job, Rasi, Photo). Contact information (phone, email, exact address) is strictly omitted.
 - *Admin Requests:* Receives the full database row.
 - **Zero-Trust Admin Authentication:** The admin panel is hidden behind a localized "3-tap" trigger on the header. It utilizes a custom `/api/auth` endpoint that validates a password and issues a secure `Bearer Token`. This token is required in the headers for any `PUT` or `DELETE` requests to succeed.
 - **XSS (Cross-Site Scripting) Sanitization:** All user-generated text is processed through a custom `escapeHTML` function before rendering. This neutralizes malicious scripts, preventing injection attacks.
 - **Pre-Flight Upload Validation:** The `/api/upload` endpoint performs strict payload inspection. It rejects any file exceeding 5MB or containing non-image MIME types, neutralizing malware uploads and protecting storage limits.
 - **Content Moderation Queue:** The database defaults all incoming payloads to a `pending` status. A profile is strictly invisible to the public API until an authenticated admin issues a `PUT` request flipping the status to `approved`.
-